KernelLeakGuard

Hybrid Static Analysis and Runtime Verification of Memory Leaks in Linux Kernel Early-Exit Paths

Om Gunjan Gupta (24BIT0316) · Abhishek Kumar Singh (24BIT0309) · Aarav Jain (24BIT0023)
Guide: Dr. Arivuselvan K, Associate Professor, SCORE
BITE303P Operating Systems Lab · Fall 2026-27 · VIT Vellore

The problem: leaks on early-exit paths

- Kernel code checks every operation and bails out early with an error code.
- Each early exit must undo everything done so far; a missing kfree() leaks memory until reboot (no garbage collector, no process exit).
- These paths run only when an allocation or device operation fails, so ordinary testing almost never executes them.
- Static tools report candidates, but many are safe: ownership moves to lists, callers, callbacks.

```
int initialize(void)
{
    char *a = kmalloc(100, GFP_KERNEL);
    if (!a)
        return -ENOMEM;
    char *b = kmalloc(200, GFP_KERNEL);
    if (!b)
        return -ENOMEM;   // a leaks
    ...
    kfree(b);
    kfree(a);
    return 0;
}
```

Leaks only when the second kmalloc() fails

Why it matters

120

previously unknown early-exit leaks found
in Linux by MLEE (USENIX ATC 2021)

weekly

kmemleak reports filed by syzbot, Google's kernel fuzzing service

0

times an error path runs in a normal test run that never fails an allocation

- **Impact:** memory pressure, degraded performance, instability; an attacker who can trigger the failure repeatedly can exhaust kernel memory (denial of service).
- **Gap:** static warnings are not proof; dynamic tools only see the paths they happen to execute and give a backtrace without the root cause.

Objectives

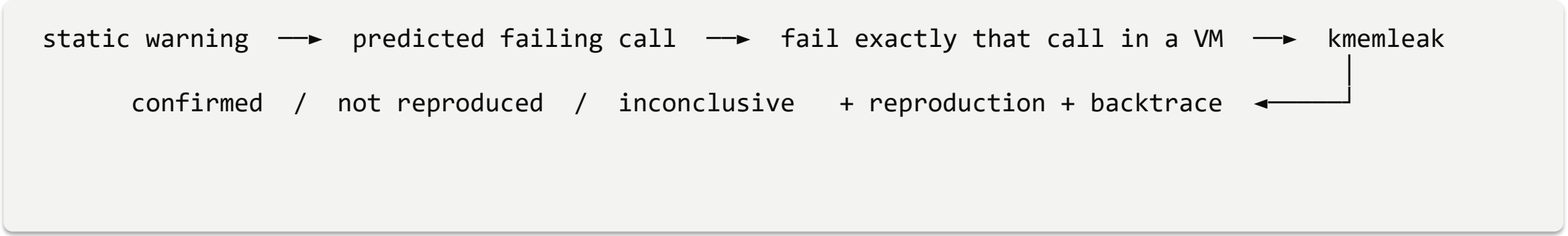
- Detect missing releases on early-exit paths with an LLVM-based static analyser (alias, ownership-transfer, interprocedural and path-correlation analysis).
- Predict, for each warning, the failing call that leads onto the leaking path.
- Build an isolated QEMU/KVM test kernel with kmemleak and per-task fault injection.
- Automatically inject the predicted failure and decide confirmed / not reproduced / inconclusive from kernel evidence.
- Evaluate on a benchmark with ground truth: static vs hybrid vs dynamic-only, ablation.
- Deliver a CLI and a dashboard with source context, evidence and suggested fixes.

Literature survey

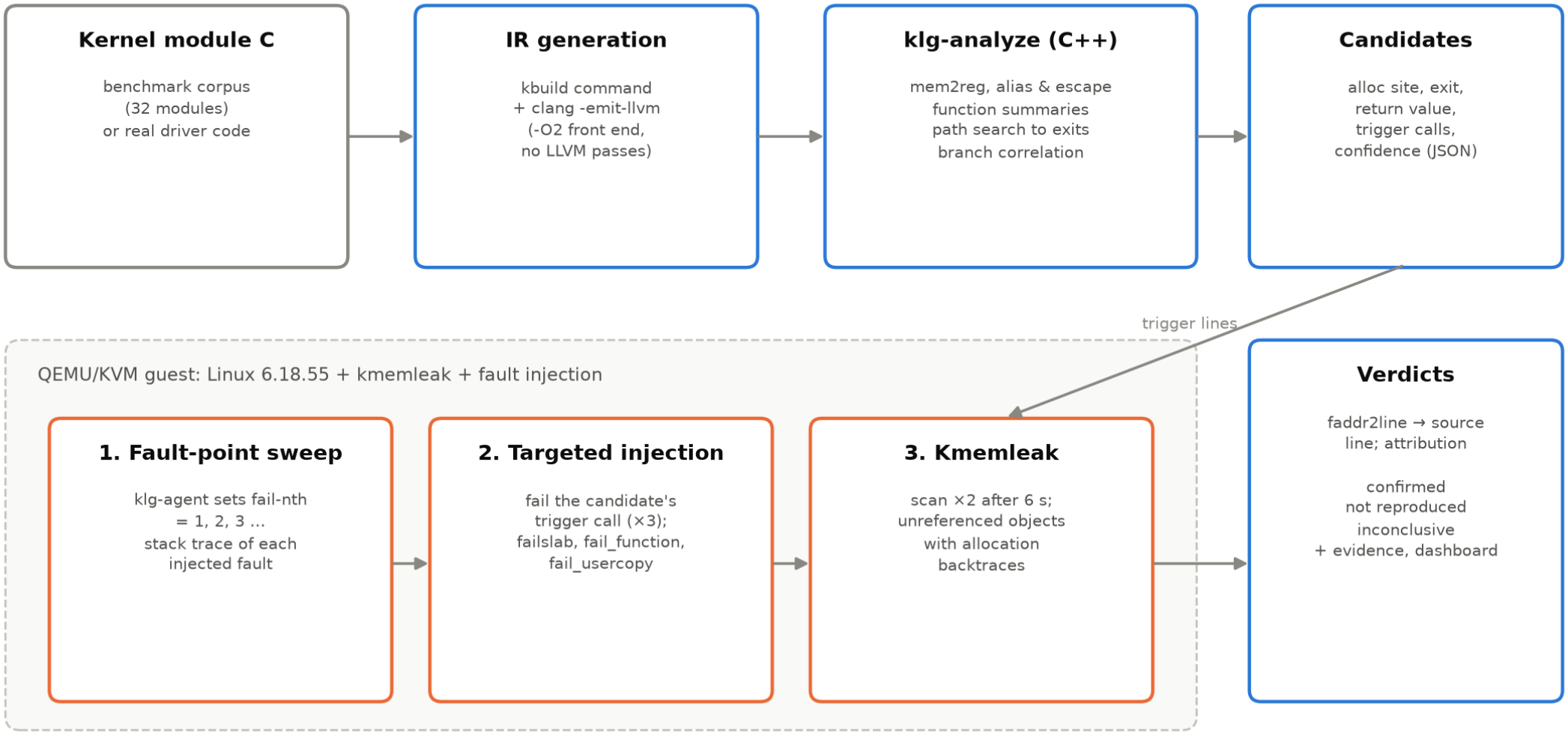
Work	Technique	Strength	Limitation
MLEE (ATC'21)	LLVM IR, early-exit vs normal path	120 new kernel leaks	static only, manual triage
Hector (DSN'13)	compare error paths	no specification needed	false positives
K-MELD (NDSS'21)	ownership reasoning	custom allocators	static only
HERO (Sec'21)	precise function pairing	many confirmed bugs	no runtime evidence
FIFUZZ (Sec'20)	context-sensitive fault injection	deep error paths	crashes, not leaks
syzkaller + kmemleak	fuzzing + fail-nth + kmemleak	real evidence	only reached paths, no root cause

Research gap and our idea

- Static detectors cover all paths but every warning needs manual triage.
- Dynamic tools give evidence only for executed paths, without saying which exit forgot what.
- No surveyed system uses the static analysis to drive targeted fault injection per warning and feeds kernel evidence back into the verdict.



System architecture



Static analyser (klg-analyze, C++ / LLVM 21)

- **IR:** re-run each module's kbuild command with -emit-llvm; mem2reg; fold configuration stubs.
- **Aliases:** casts, GEPs, PHIs, and fields of local structs/arrays.
- **Handled =** kfree & friends, or ownership transfer (store to non-local memory, return, capturing call).
- **Summaries:** frees / captures / returns each parameter; allocation wrappers.
- **Path search:** DFS to every return, skip the alloc-failed edge and contradictory branches (correlation), loops ×2.
- **Report:** exit, returned errno, failing trigger calls, confidence.

```
{
 "function": "sensor_setup",
 "allocator": "kmalloc",
 "alloc": {"line": 32},
 "exit": {"line": 54},
 "return_value": "-ENOMEM",
 "triggers": [
   {"callee": "kasprintf",
    "line": 38}],
 "confidence": "high"
}
```

Runtime verifier (QEMU/KVM guest, Linux 6.18.55)

- **Fault-point sweep:** klg-agent writes N to /proc/thread-self/fail-nth (N = 1, 2, 3 ...) and runs the routine; the stack trace of each injected fault gives N → source line.
- **Targeted injection:** for each warning, inject the N that fails its trigger call, 3 times (failslab, fail_function via ftrace kprobes, fail_usercopy).
- **Kmemleak:** wait 6 s, scan twice; attribute each unreferenced object by its allocation backtrace (faddr2line) or, for unloaded init code, by allocation size.
- **Trustworthy evidence:** no stale pointer copies: page-allocator stacks, KSTACK_ERASE, no task-stack scanning, init_on_alloc / init_on_free for slab memory.
- **Speed:** a guest boots in ~2 s; both VM runs take about 2 min.

Operating-system concepts in the project

Concept	Where
Memory management	slab allocator, kmalloc caches, vmalloc, ownership, kmemleak mark & scan
Error handling / faults	errno propagation, goto cleanup, failslab / fail_function / fail_usercopy
Processes & system calls	per-task fault state (fail-nth), task stacks, write / finit_module / pread
Device management	loadable modules, misc devices, platform drivers, devm memory
Synchronisation	mutex-protected lists, RCU deferred free, kref reference counting
Virtualisation	QEMU + KVM, virtio-9p sharing, initramfs boot
Performance analysis	analysis time, VM time, executions, precision / recall

Benchmark corpus (dataset)

32

kernel modules
(17 leaky, 15 leak-free)

17

leaking allocation sites

42

safe allocation sites

- Patterns from real kernel bugs: wrong goto label, copy_from_user failure, loops, wrappers, registration failures, conditional allocation, kmem_cache, vmalloc.
- Tricky safe code: list ownership, caller-owned fields, devm, kref, RCU, factories, correlated conditions, rollback loops.
- Ground truth in the source: every allocation is annotated /* GT:LEAK */ or /* GT:SAFE */.

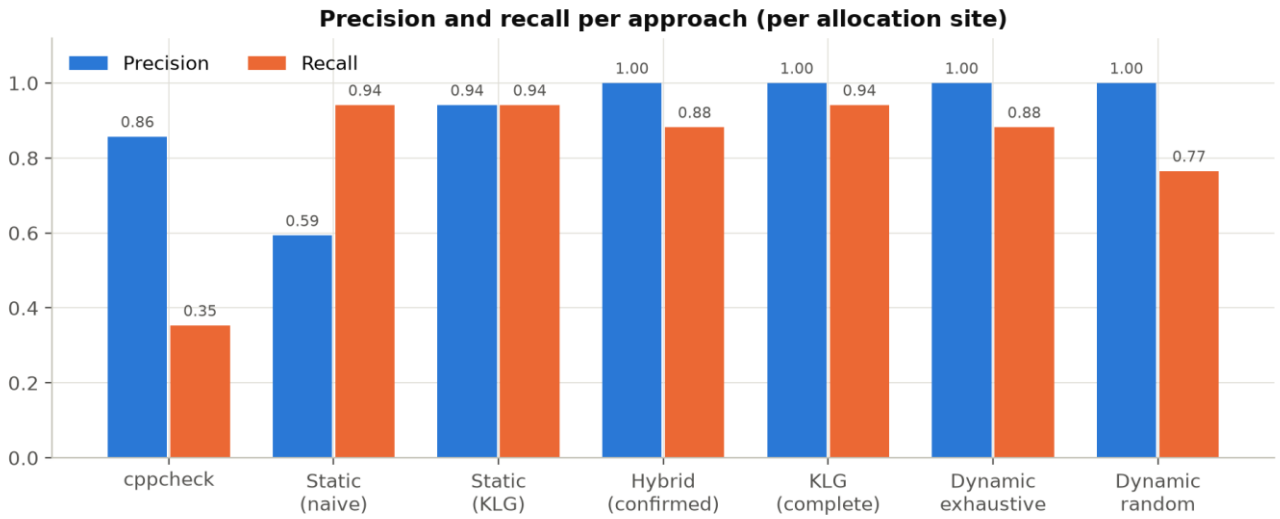
Implementation and module division

Member	Modules
Om Gunjan Gupta (24BIT0316)	Static analyser (C++/LLVM): IR preparation, alias/escape, summaries, path search
Abhishek Kumar Singh (24BIT0309)	Guest kernel & VM, klg-agent fault-injection driver (C), kmemleak protocol
Aarav Jain (24BIT0023)	Corpus & ground truth, Python verifier & evaluation, Streamlit dashboard

- C++17 + LLVM/Clang 21 · C (static) · Python 3 · Linux 6.18.55 LTS · QEMU/KVM on WSL2
- Single command: python3 -m klg all → build, analyse, verify in VM, evaluate

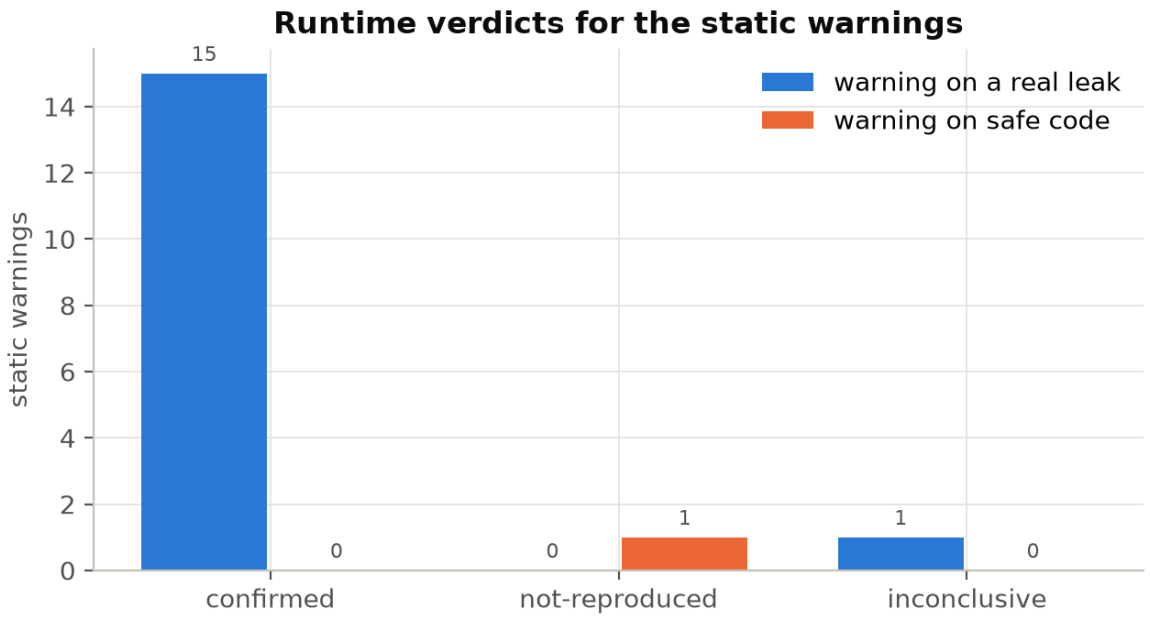
Results: detection quality

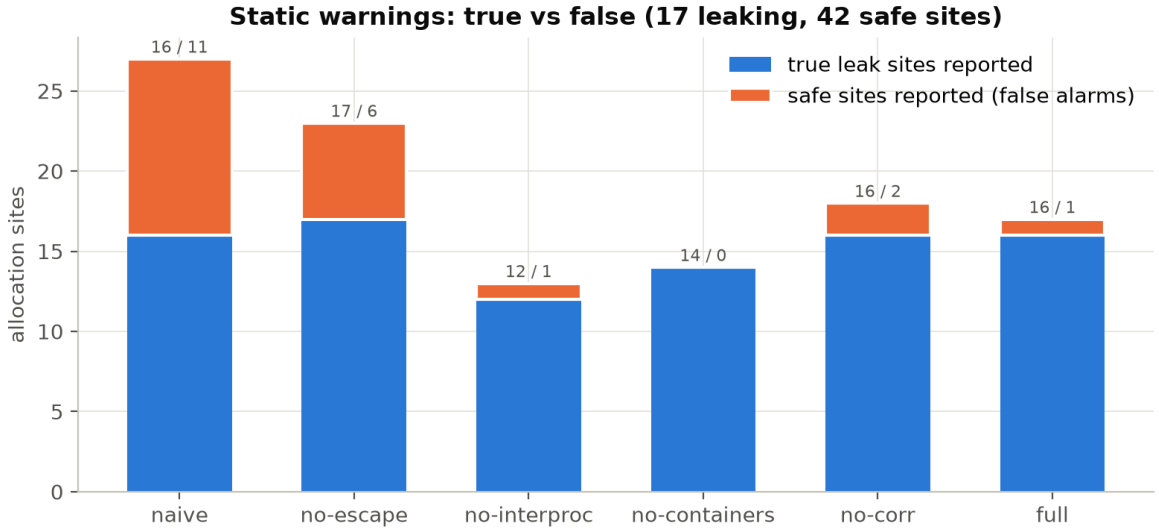
Approach	TP	FP	FN	Prec.	Rec.
cppcheck 2.19	6	1	11	0.86	0.35
Static, naive	16	11	1	0.59	0.94
Static, KLG	16	1	1	0.94	0.94
Hybrid (confirmed)	15	0	2	1.00	0.88
KLG complete	16	0	1	1.00	0.94
Dynamic, exhaustive	15	0	2	1.00	0.88
Dynamic, random	13	0	4	1.00	0.77



KernelLeakGuard: 16 of 17 leaks, no false positive; every confirmed leak has kernel evidence.

Runtime verdicts and ablation





- **15 confirmed, 1 not reproduced, 1 inconclusive** — no false confirmation
- **Without ownership transfer:** precision 0.74; naive: 11 false alarms

Real Linux 6.18.55 kernel code

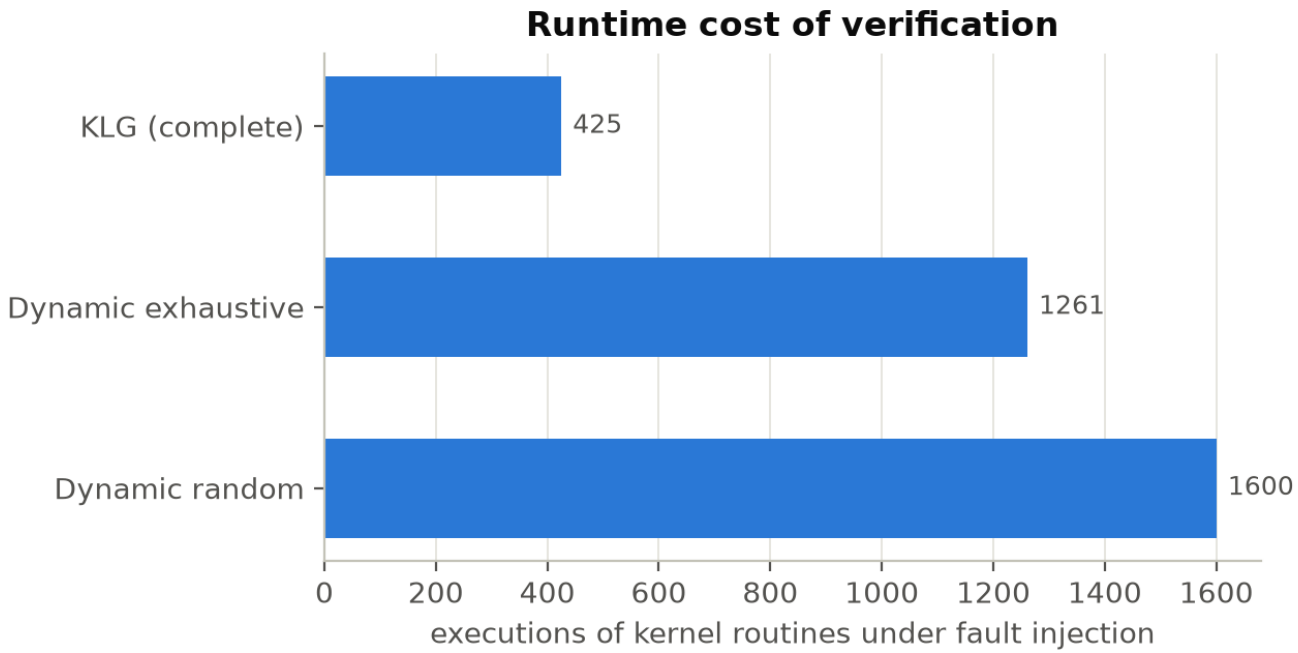
- **Static:** 39 modules, 6376 functions, 518 allocation sites → 11 warnings (115 before general refinements: ERR_PTR, ownership of published objects, infeasible cleanup paths; benchmark unchanged)
- **Runtime (VM):** 39 modules, 9779 fault points of module init exercised; no warning confirmed
- **Re-introduced leaks:** bonding bond_create() (real bug, fixed upstream by 8d6184e4881b) → detected, confirmed by kmemleak in the VM, fixed; null_blk null_alloc_dev() (seeded) → detected, confirmed by kmemleak in the VM, fixed
- **SLUB leak (root-caused):** init of erofs, f2fs, hfsplus, nilfs2 exposed a slab-cache leak in mm/slub.c when sysfs registration fails; still in mainline; our patch: 20 → 0 leaked objects in the VM
- **f2fs regression (runtime):** kobject names leak since commit b635f2ecdb (Apr 2026, also in stable); our patch: 29 → 0 leaks
- **Static finds:** ntfs3 do_action() leaks an mft_inode when mi_format_new() fails; ufs ubh_bforget() never frees the ufs_buffer_head (reported 2018, unfixed); both in mainline, patches remove the warnings
- **null_blk crash (root-caused):** use-after-free of module memory: deferred fput of the partition-scan file runs after a failed init freed the module
- **Lesson:** static precision drops on real code; runtime verification keeps reports clean and its fault injection finds real kernel defects

Case study: klg_l02 — wrong goto label

```
tx = kmalloc(TX_SIZE, GFP_KERNEL);   // 32
if (!tx) {
    ret = -ENOMEM;
    goto err_rx;
}
name = kasprintf(GFP_KERNEL, ...);  // 38
if (!name) {
    ret = -ENOMEM;
    goto err_rx;   // BUG: err_tx
}
...
err_tx:
    kfree(tx);
err_rx:
    kfree(rx);
    return ret;
```

- **Static:** kmalloc at 32 leaks when returning -ENOMEM; trigger kasprintf (38).
- **Sweep:** fault #5 fails sensor_setup+0x5f/0xb0.
- **Confirm:** injected 3×, kmemleak: 3 unreferenced objects allocated at line 32.
- **Verdict:** confirmed. Fix: goto err_tx.

Performance



- **Static analysis:** 14 ms for 32 modules
- **Mapping + confirmation VM runs:** 0 s + 94 s
- **KernelLeakGuard:** 425 executions in total (module-filtered sweep + confirmation) vs 1261 exhaustive
- **Fix patches:** 14 of 15 validated (compile, re-analysis, VM re-run)
- **Hardware:** laptop, Ryzen 7, 15 GB, WSL2 + KVM

Demonstration

```
cd ~/KernelLeakGuard
python3 -m klg all
python3 -m klg show

.venv/bin/streamlit run dashboard/app.py
# → http://localhost:8501
```

- Pipeline: build → analyse → VM sweep → targeted injection → kmemleak → verdicts
- Dashboard: findings with verdicts; source with allocation / failing call / exit highlighted; kmemleak backtraces; evaluation charts
- Live: introduce a bug in a module, rerun, watch it get confirmed

Conclusion and future work

- Static analysis finds early-exit leaks with F1 0.94 (naive: 0.73).
- Targeted fault injection + kmemleak confirms them with evidence; precision 1.00.
- Complete tool: 16/17 leaks, 0 false positives, validated fix patches for 14/15.
- Predicting the failing call + module-filtered injection: cheaper than blind injection.
- kmemleak must be protected from stale pointer copies (stacks, reused slab memory).

Future work

- Field-sensitive heap ownership (nested leaks)
- Real kernel code: 39 hardware-free modules already built into the guest
- Reproduce MLEE's historical leaks
- Loop-counter reasoning; patches for shared error labels
- CI integration (KernelCI)

Thank you

Questions?

Om Gunjan Gupta · Abhishek Kumar Singh · Aarav Jain